

PRACTICAL - 2

AIM:RSA Encryption and Decryption

Implement the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations.

CLI:

CODE:

```
from math import gcd

# Function to find modular inverse
def mod_inverse(e, phi):
    for d in range(2, phi):
        if (d * e) % phi == 1:
            return d
    return None

# Input two prime numbers
p = int(input("Enter first prime number (p): "))
q = int(input("Enter second prime number (q): "))

# Calculate n and phi
n = p * q
phi = (p - 1) * (q - 1)

# Choose e
e = int(input("Enter value of e: "))

if gcd(e, phi) != 1:
    print("e must be co-prime with phi(n).")
    exit()

# Calculate d
d = mod_inverse(e, phi)

print("\nPublic Key :", (e, n))
print("Private Key:", (d, n))

# Encrypt
msg = int(input("\nEnter message (integer less than n): "))
cipher = pow(msg, e, n)
print("Encrypted Message:", cipher)

# Decrypt
plain = pow(cipher, d, n)
print("Decrypted Message:", plain)
```

SUMEET YADAV

T125

SHETH L.U.J AND SIR M.V. COLLEGE

SUBJECT: CYBERSECURITY

OUTPUT:

```
IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\info\OneDrive\Desktop\CyberSecurity_Prac2.py =====
Enter first prime number (p): 11
Enter second prime number (q): 13
n = 143
phi = 120
Enter value of e: 7

Public Key : (7, 143)
Private Key: (103, 143)

Enter message (integer less than n): 9
Encrypted Message: 48
Decrypted Message: 9
>>>
```

```
C:\WINDOWS\py.exe
Enter first prime number (p): 11
Enter second prime number (q): 13
n = 143
phi = 120
Enter value of e: 7

Public Key : (7, 143)
Private Key: (103, 143)

Enter message (integer less than n): _
```

SUMEET YADAV
T125

GUI:

CODE:

```
import tkinter as tk
from tkinter import messagebox
from math import gcd

# Function to find modular inverse
def mod_inverse(e, phi):
    for d in range(2, phi):
        if (d * e) % phi == 1:
            return d
    return None

def generate_keys():
    global n, e, d

    try:
        p = int(entry_p.get())
        q = int(entry_q.get())
        e = int(entry_e.get())

        n = p * q
        phi = (p - 1) * (q - 1)

        if gcd(e, phi) != 1:
            messagebox.showerror("Error", "e must be co-prime with phi(n)")
            return

        d = mod_inverse(e, phi)

        lbl_public.config(text=f"Public Key: ({e}, {n})")
        lbl_private.config(text=f"Private Key: ({d}, {n})")

    except:
        messagebox.showerror("Error", "Enter valid numbers.")

def encrypt():
    try:
        msg = int(entry_msg.get())

        if msg >= n:
            messagebox.showerror("Error", f"Message must be less than {n}")
            return

        cipher = pow(msg, e, n)
```

SHETH L.U.J AND SIR M.V. COLLEGE

SUBJECT: CYBERSECURITY

```
entry_cipher.delete(0, tk.END)
entry_cipher.insert(0, str(cipher))

except:
    messagebox.showerror("Error", "Generate keys first.")

def decrypt():
    try:
        cipher = int(entry_cipher.get())
        plain = pow(cipher, d, n)

        entry_plain.delete(0, tk.END)
        entry_plain.insert(0, str(plain))

    except:
        messagebox.showerror("Error", "Invalid Cipher Text.")

#GUI
root = tk.Tk()
root.title("RSA Encryption & Decryption")
root.geometry("450x400")

tk.Label(root, text="RSA Encryption & Decryption",
        font=("Arial", 16, "bold")).pack(pady=10)

frame = tk.Frame(root)
frame.pack()

tk.Label(frame, text="Prime p").grid(row=0, column=0, padx=5, pady=5)
entry_p = tk.Entry(frame)
entry_p.grid(row=0, column=1)

tk.Label(frame, text="Prime q").grid(row=1, column=0, padx=5, pady=5)
entry_q = tk.Entry(frame)
entry_q.grid(row=1, column=1)

tk.Label(frame, text="Public Key e").grid(row=2, column=0, padx=5, pady=5)
entry_e = tk.Entry(frame)
entry_e.grid(row=2, column=1)

tk.Button(root, text="Generate Keys",
        command=generate_keys,
        bg="lightblue").pack(pady=10)

lbl_public = tk.Label(root, text="Public Key:")
lbl_public.pack()
```

SUMEET YADAV

T125

```
lbl_private = tk.Label(root, text="Private Key:")  
lbl_private.pack()
```

```
tk.Label(root, text="Message").pack()  
entry_msg = tk.Entry(root)  
entry_msg.pack()
```

```
tk.Button(root, text="Encrypt",  
          command=encrypt,  
          bg="lightgreen").pack(pady=5)
```

```
tk.Label(root, text="Cipher Text").pack()  
entry_cipher = tk.Entry(root)  
entry_cipher.pack()
```

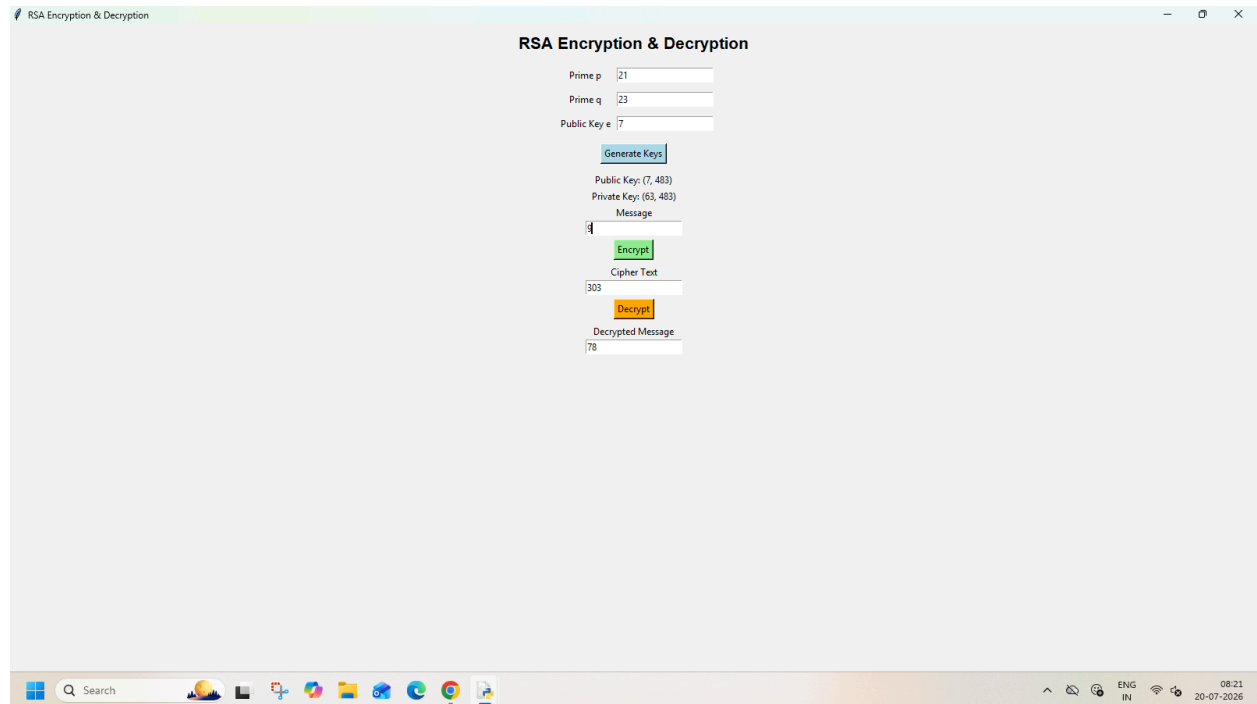
```
tk.Button(root, text="Decrypt",  
          command=decrypt,  
          bg="orange").pack(pady=5)
```

```
tk.Label(root, text="Decrypted Message").pack()  
entry_plain = tk.Entry(root)  
entry_plain.pack()
```

```
root.mainloop()
```

SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT: CYBERSECURITY

OUTPUT:



The screenshot shows a web application titled "RSA Encryption & Decryption" running in a browser window. The application interface includes the following elements:

- Prime p:** Input field with value 21.
- Prime q:** Input field with value 23.
- Public Key e:** Input field with value 7.
- Generate Keys:** A button to generate the RSA key pair.
- Public Key:** Displayed as (7, 483).
- Private Key:** Displayed as (63, 483).
- Message:** Input field with value 4.
- Encrypt:** A green button to encrypt the message.
- Cipher Text:** Output field showing the encrypted value 303.
- Decrypt:** An orange button to decrypt the cipher text.
- Decrypted Message:** Output field showing the original value 78.

The application is running on a Windows operating system, as indicated by the taskbar at the bottom, which shows the Start button, search bar, and various application icons. The system clock in the bottom right corner displays the time 08:21 and date 20-07-2026.

SUMEET YADAV
T125